

Indice

- Capitolo 1: Analisi
 - 1.1 Descrizione e requisiti
 - 1.1.1 Requisiti funzionali
 - 1.1.2 Requisiti non funzionali
 - 1.2 Modello del Dominio
- Capitolo 2: Design
 - 2.1 Architettura
 - 2.2 Design dettagliato
 - 2.2.1 Daniel Patryk Bak
 - Gestione dei pannelli
 - Leaderboard
 - 2.2.2 Giacomo Mengozzi
 - Superfici avanzate
 - Fisica della pallina e integrazione Model–Controller
 - 2.2.3 Federico Sparvoli
 - Input del colpo
 - Ciclo di vita del match e progressione delle mappe
 - 2.2.4 Mattia D'Ambrosio
 - Fisica, geometria degli ostacoli e gestione vettoriale
 - Pro e Contro
 - Creazione degli ostacoli avanzati (Appiccicosi, Rimbalzanti e Portali)
 - Architettura e separazione Vista-Logica
 - Pro e Contro
- Capitolo 3: Sviluppo
 - 3.1 Testing automatizzato
 - 3.1.1 Daniel Patryk Bak
 - 3.1.2 Giacomo Mengozzi
 - 3.1.3 Federico Sparvoli
 - 3.1.4 Mattia D'Ambrosio
 - 3.2 Note di sviluppo
 - 3.2.1 Daniel Patryk Bak
 - 3.2.2 Giacomo Mengozzi
 - 3.2.3 Federico Sparvoli
 - 3.2.4 Mattia D'Ambrosio
- Capitolo 4: Commenti finali

- 4.1 Autovalutazione e lavori futuri
 - 4.1.1 Daniel Patryk Bak
 - 4.1.2 Giacomo Mengozzi
 - 4.1.3 Federico Sparvoli
 - 4.1.4 Mattia D'Ambrosio
- 4.2 Difficoltà incontrate e commenti per i docenti
- Appendice A
 - Guida Utente
- Appendice B
 - Esercitazioni di laboratorio

Capitolo 1: Analisi

1.1 Descrizione e requisiti

Il software è un gioco di mini-golf in due dimensioni per uno o più giocatori in modalità a turni. Il giocatore controlla una pallina su una mappa con ostacoli e superfici di diverso tipo, con l'obiettivo di farla entrare nella buca nel minor numero di colpi possibile. Il sistema di controllo è intuitivo: ogni colpo viene effettuato trascinando il mouse dalla pallina in direzione opposta a quella desiderata, calibrando così potenza e traiettoria. Il gioco è strutturato in una sequenza di mappe (partendo da un tutorial iniziale seguito da livelli in ordine casuale), al termine delle quali i punteggi vengono salvati in una classifica globale.

1.1.1 Requisiti funzionali

- Il giocatore effettua un colpo trascinando il mouse dalla pallina: la direzione e la potenza sono determinate dalla posizione del cursore rispetto alla pallina, un indicatore visivo mostra in tempo reale direzione e potenza del colpo durante il trascinamento.
- Il gioco supporta più giocatori in modalità a turni.
- La pallina interagisce con ostacoli di forme diverse e superfici con proprietà fisiche differenti.
- Al completamento di ogni mappa viene mostrata una classifica intermedia con i colpi effettuati da ciascun giocatore.

1.1.2 Requisiti non funzionali

- Il gioco prevede superfici e ostacoli con proprietà fisiche avanzate, tra cui terreni e ostacoli che rallentano, velocizzano o modificano la traiettoria.

- È possibile salvare la partita in corso e riprenderla in un secondo momento.
- La classifica finale è persistente tra sessioni diverse e si aggiorna al termine di ogni partita completata.

1.2 Modello del Dominio

Una partita (*Match*) di mini-golf è composto da una sequenza di mappe (*Map*), ognuna costituita da una pallina (*Ball*), una buca (*Hole*), una o più superfici (*Surface*) ed alcuni ostacoli (*Obstacle*). La pallina si muove nella mappa interagendo con superfici e ostacoli che ne modificano velocità e direzione secondo leggi fisiche. Una partita coinvolge uno o più giocatori (*Player*) che si alternano a turni per colpire la pallina, indicandone direzione e potenza, e mandandola in buca. Il punteggio di ciascun partecipante su una mappa corrisponde al numero di colpi effettuati.

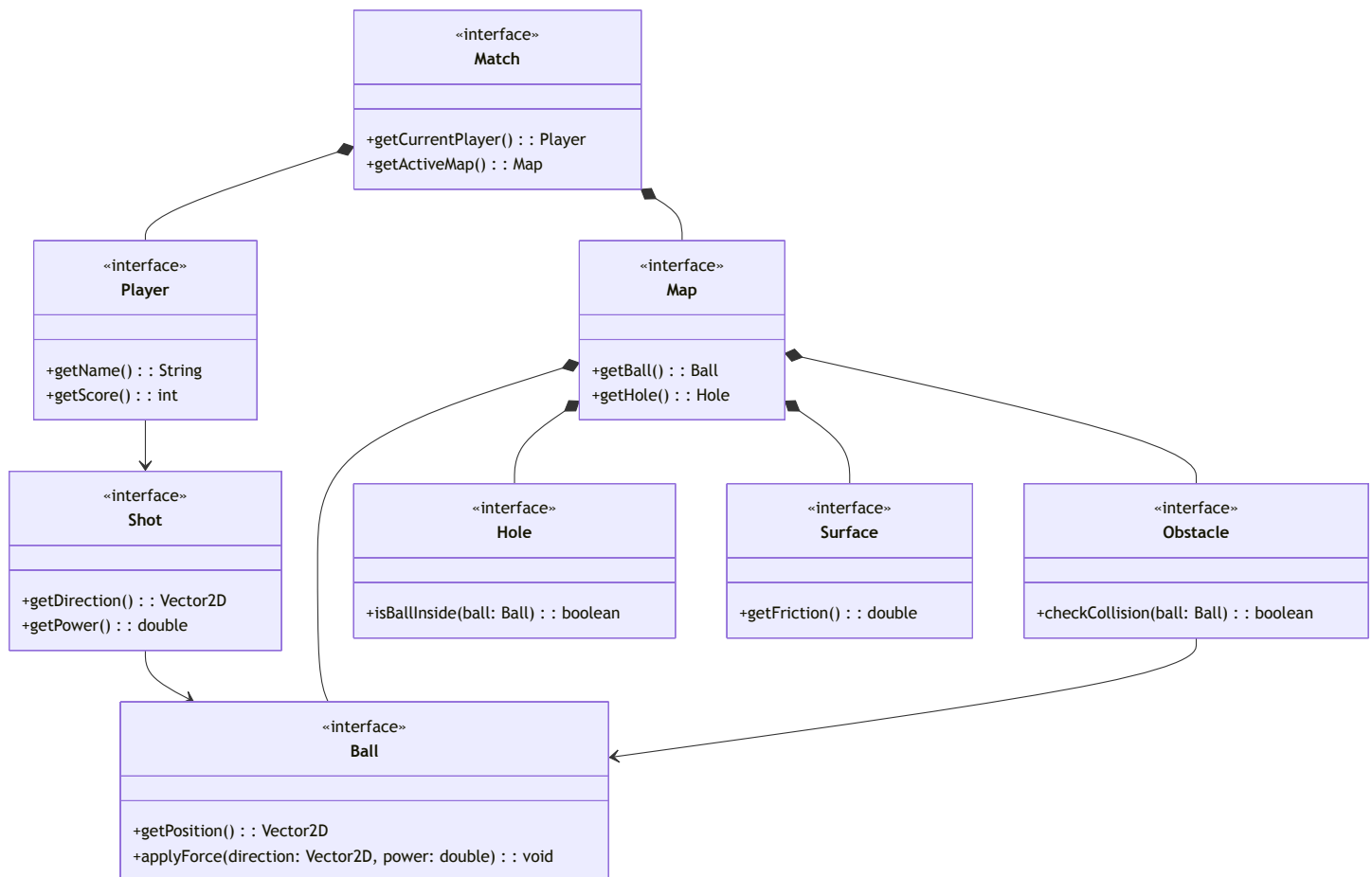


Figura 1: Schema UML dell'analisi del dominio, con rappresentate le entità principali ed i rapporti fra loro.

Capitolo 2: Design

2.1 Architettura

Il software segue il pattern architetturale MVC (Model-View-Controller).

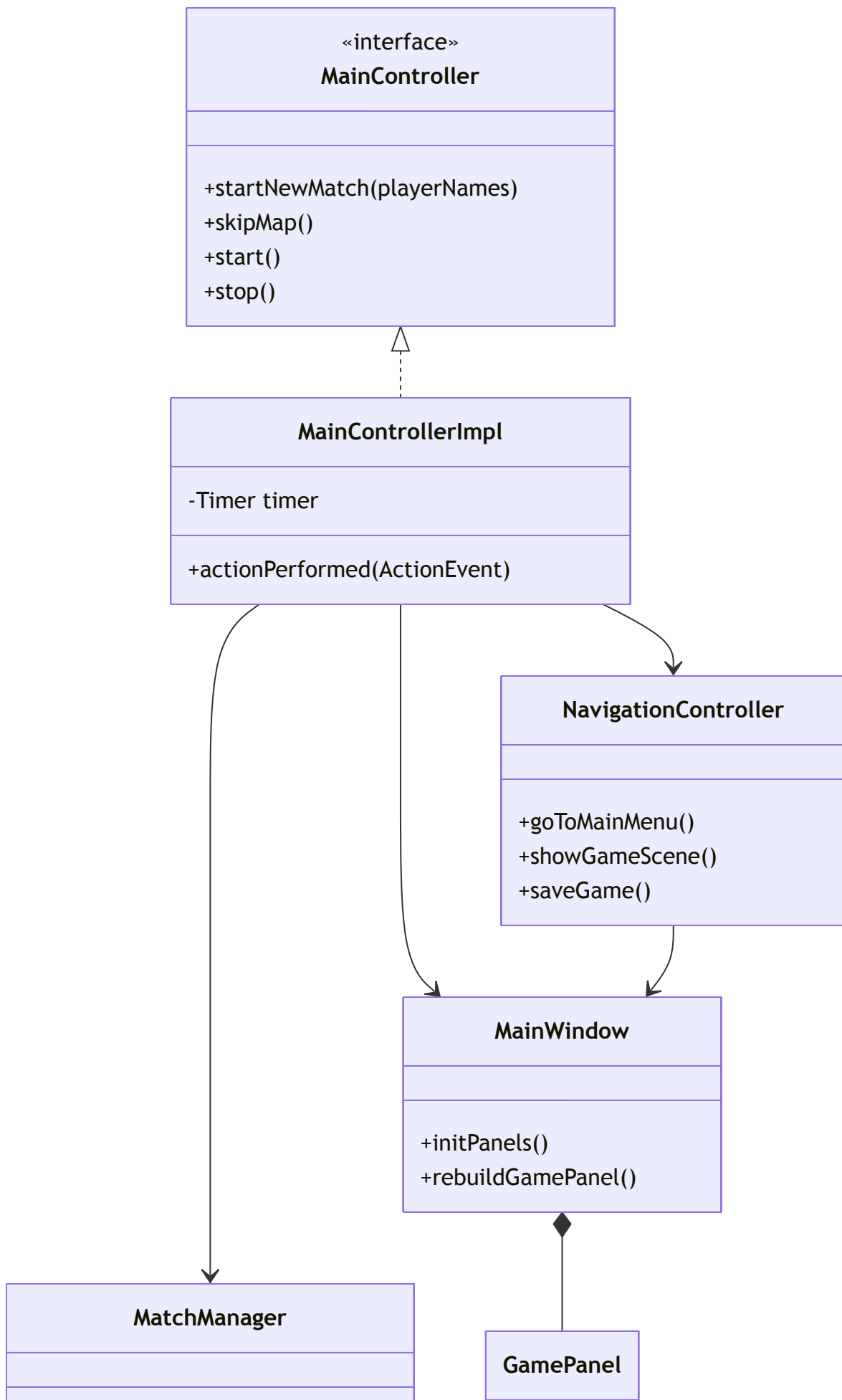
Model: contiene lo stato del gioco e le regole, senza alcuna dipendenza dalla view o dal controller. Le entità principali sono `GameMap` (la mappa corrente con pallina, buca, superfici e ostacoli), `GameState` (lo stato del turno: giocatore attivo, contatore colpi, movimento della pallina) e `ShotState` (lo stato del colpo in corso: direzione, potenza, posizione della pallina).

Controller: coordina model e view senza che i due si conoscano direttamente. La logica di controllo è strutturata gerarchicamente per separare nettamente le responsabilità:

- `MainController` : Inizializza i componenti principali e ospita il Game Loop tramite un Timer.
- `MatchManager` : agisce da supervisore della sessione di gioco. Inizializza le partite e gestisce le transizioni logiche tra una buca e l'altra.
- `GameController` : gestisce il ciclo di vita della singola mappa attiva. Ad ogni tick ricevuto dal `MainController`, aggiorna la simulazione fisica della pallina, verifica le condizioni di vittoria (ingresso in buca) e delega l'avanzamento dei turni.
- `NavigationController` : si occupa di gestire le transizioni tra i vari pannelli della view nella `MainWindow` .

View: `MainWindow` ospita i pannelli tramite un `CardLayout`; `GamePanel` compone la mappa di gioco (`MapPanel`) e l'overlay di input (`ShotViewPanel`).

Il controller non dipende mai dalla view concreta: comunica con essa unicamente tramite interfacce strette e callback funzionali (`Runnable`, `Consumer`, `Supplier`). Ciò implica che sostituire Swing con un'altra libreria grafica (ad esempio `JavaFX`) non richiederebbe alcuna modifica al controller né al model: sarebbe sufficiente riscrivere i pannelli della view e ricablare i callback al momento della costruzione.



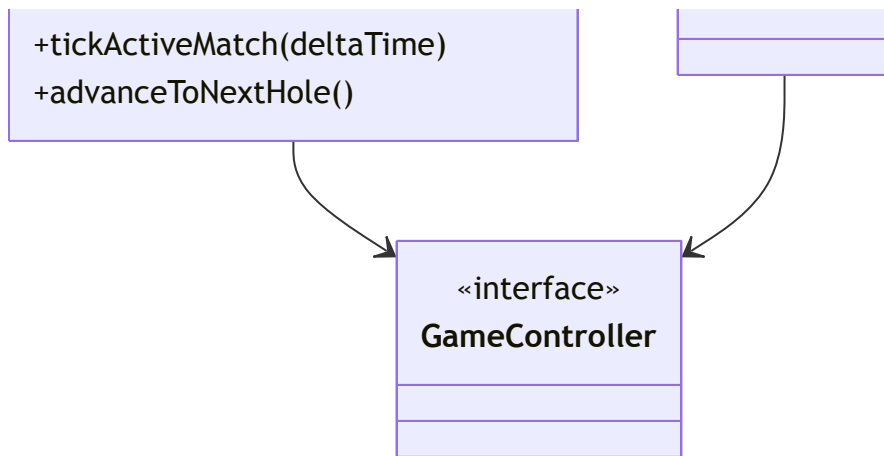


Figura 2: Schema UML del design dell'architettura, con rappresentate le classi principali ed i rapporti fra loro.

2.2 Design dettagliato

2.2.1 Daniel Patryk Bak

Gestione dei pannelli

Problema

Come ogni gioco, anche mini-gOOLF necessita di un menu principale da cui poter transitare tra diverse "scene", in questo caso i pannelli. È fondamentale rispettare il pattern MVC, assegnando ad un controller la gestione delle transizioni dei pannelli (view).

Soluzione

Per poter rendere questa transizione il più semplice e fluida possibile, si è scelto il `CardLayout`, mentre per pannelli che devono apparire soltanto in sovrapposizione (quali `PausePanel` e `MidLeaderBoardPanel`) si è scelto il metodo `Glasspane` della libreria `Swing`. I pannelli vengono illustrati all'interno della `MainWindow` ovvero la finestra dell'intera applicazione.

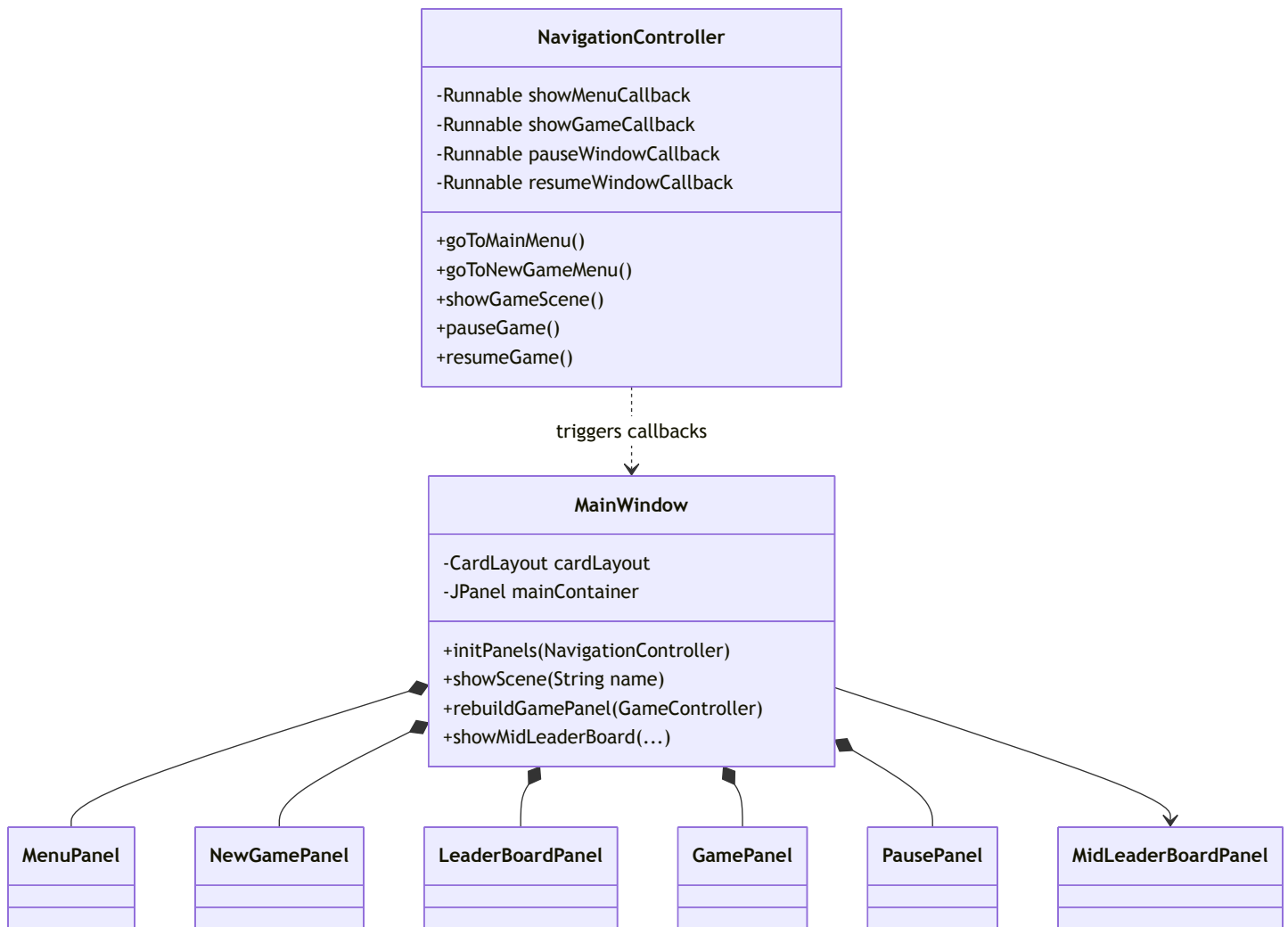


Figura 3: Schema UML dei pannelli e di come sono gestiti.

Pro e Contro

Pro:

- Disaccoppiamento netto: Il `NavigationController` gestisce la logica di transizione senza mai possedere un riferimento diretto alla `MainWindow` o ai pannelli. La comunicazione avviene esclusivamente tramite callback funzionali (`Runnable` e `Consumer`), garantendo che il controller non possa manipolare impropriamente lo stato interno della View.
- Efficienza del `CardLayout`: I pannelli statici (`Menu`, `New Game`, `Leaderboard`) vengono istanziati una sola volta e mantenuti in memoria. Il `CardLayout` si limita a nascondarli o mostrarli, rendendo le transizioni di scena istantanee.
- Gestione nativa degli Overlay: L'utilizzo del `GlassPane` per il `PausePanel` e il `MidLeaderBoardPanel` permette di sovrapporre interfacce trasparenti al gioco in esecuzione senza dover alterare la gerarchia dei componenti del `GamePanel`.

Contro:

- Limitazione strutturale del GlassPane: La libreria Swing consente a un JFrame di avere un solo GlassPane attivo per volta. Questo costringe ad implementare una logica di salvataggio e ripristino manuale (come visibile nel metodo `showMidLeaderBoard`) nel caso in cui più overlay debbano susseguirsi o sovrapporsi.
- Prolissità del costruttore: L'estrazione di tutti i comportamenti della finestra sotto forma di callback rende il costruttore del NavigationController leggermente prolisso e dipendente dal corretto "cablaggio" iniziale.

Leaderboard

Problema

Per poter misurare le proprie abilità in modo tangibile, un sistema di classifica è quello più efficace. Il problema, dal punto di vista progettuale, risiede nel separare la logica di calcolo e persistenza dei punteggi dalla loro rappresentazione visiva, garantendo al contempo due visualizzazioni distinte: un riepilogo temporaneo alla fine di ogni singola buca e una classifica globale, persistente su file, accessibile dal menu principale.

Soluzione

Si è deciso di suddividere la responsabilità in cinque componenti chiave, definendo confini netti per rispettare il pattern MVC e il principio di singola responsabilità:

- `LeaderBoardManager` (Model): Gestisce la persistenza dei dati su un file di testo locale (`leaderboard.txt`). Custodisce la logica di dominio per l'aggiornamento dei record storici, assicurandosi di sovrascrivere il punteggio di un giocatore solo se il nuovo risultato è strettamente inferiore (migliore) del precedente.
- `MatchManager` (Controller): Orchestra il flusso della partita attiva. Al termine di ogni singola buca, è lui a raccogliere la mappa non persistente con i punteggi parziali dei giocatori e a coordinare l'apertura del riepilogo grafico temporaneo.
- `MidLeaderBoardPanel` (View): Funge da pannello in sovraimpressione (overlay). Riceve i dati temporanei dal `MatchManager`, li ordina dinamicamente in memoria e li mostra a schermo per dare un feedback immediato, mettendo in pausa il gioco in attesa del passaggio alla mappa successiva.
- `NavigationController` (Controller): Gestisce il routing dell'intera applicazione e fa da "ponte" per la classifica globale. Quando l'utente richiede di visualizzare i punteggi dal menù, questo controller interroga il `LeaderBoardManager` per estrarre lo storico e inietta i dati pronti per la lettura nella scena grafica.
- `LeaderBoardPanel` (View): È una scena dedicata ospitata all'interno del `CardLayout`. Riceve i dati storici dal `NavigationController`, si occupa del loro ordinamento finale e li rappresenta visivamente in una griglia, mettendo in risalto le prime posizioni.

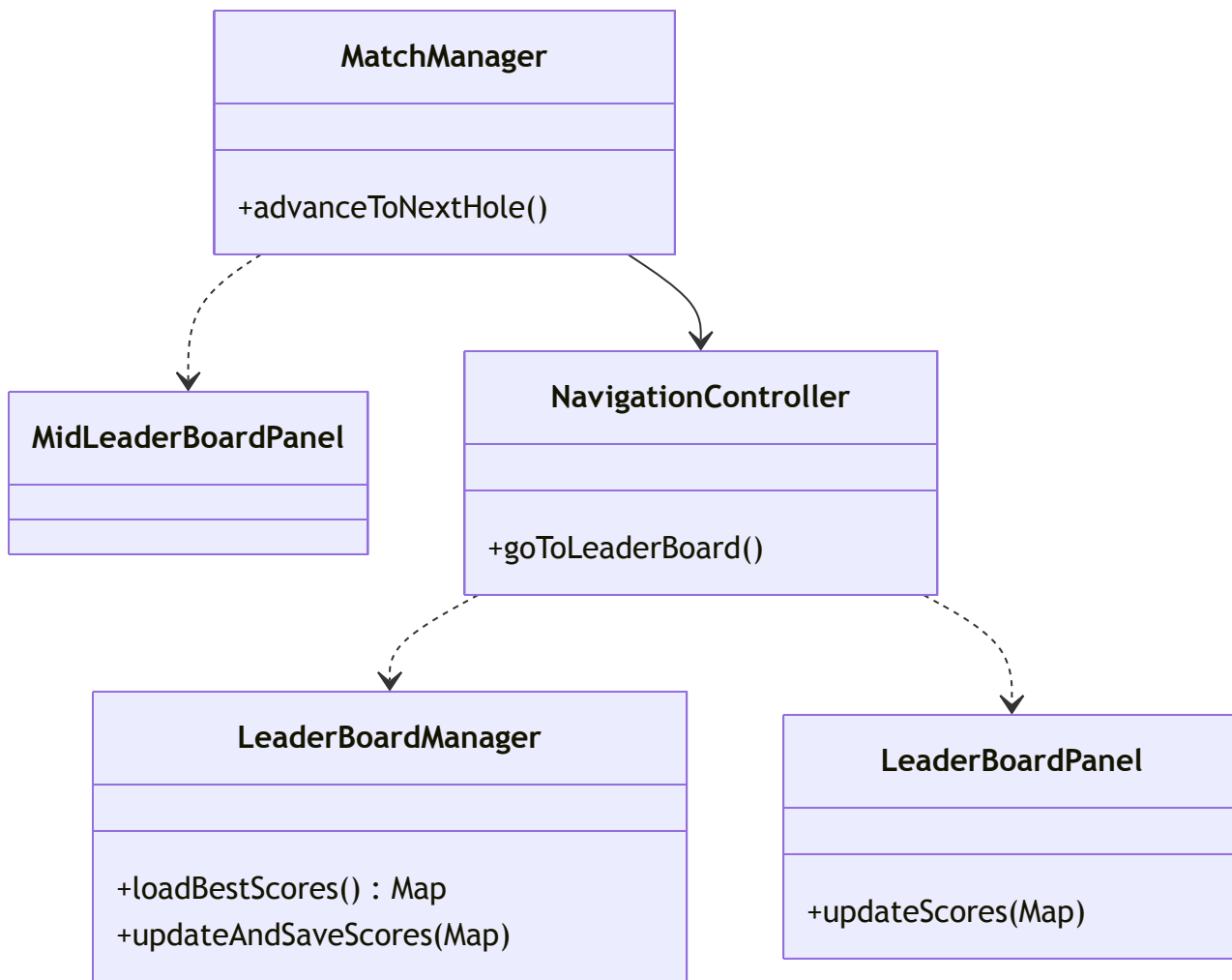


Figura 4: Schema UML di come è stato implementato il sistema della classifica.

Pro e Contro

Pro:

- Resilienza e Semplicità: L'utilizzo di un file di testo semplice (.txt) con codifica Chiave:Valore per il salvataggio rende il sistema di I/O rapido, robusto e facilmente ispezionabile senza la necessità di includere librerie di database esterne.
- Riutilizzo del layout: Entrambi i pannelli grafici (`LeaderBoardPanel` e `MidLeaderBoardPanel`) si affidano a un `GridLayout` dinamico che scala automaticamente le righe in base al numero di giocatori in mappa o registrati nel file, garantendo consistenza visiva.

Contro:

- Ordinamento ripetuto: Attualmente, l'ordinamento della mappa (conversione in `List<Map.Entry>` e successiva sort) viene delegato alla View (`LeaderBoardPanel` e `MidLeaderBoardPanel`) al momento del rendering. A livello architetturale "purista", sarebbe stato preferibile che il Model

restituisse una struttura dati già ordinata (es. una lista di Record), sollevando l'interfaccia grafica da calcoli di manipolazione dati.

2.2.2 Giacomo Mengozzi

Superfici avanzate

Problema

Il gioco richiede superfici con proprietà fisiche diverse (attrito, accelerazione, vento) che influenzano il moto della pallina. Le 4 superfici base (erba, sabbia, ghiaccio, terra) differiscono solo per il coefficiente di attrito, mentre le superfici avanzate (`BoostSurface` , `WindySurface`) alterano la velocità con modalità diverse. È necessario aggiungere questi comportamenti senza duplicare la logica già presente nelle superfici base.

Soluzione

È stato adottato il **Decorator Pattern**: l'interfaccia `Surface` è implementata sia da `ShapedSurface` sia da `AbstractSurfaceDecorator` (che incapsula `Surface` e ne delega per default tutti i metodi). `BoostSurface` e `WindySurface` estendono il decoratore astratto sovrascrivendo rispettivamente `getFriction()` e `getWind()`. Il motore fisico (`BasicFrictionStrategy`) interroga la superficie corrente tramite questi metodi, senza conoscere se sia base o decorata.

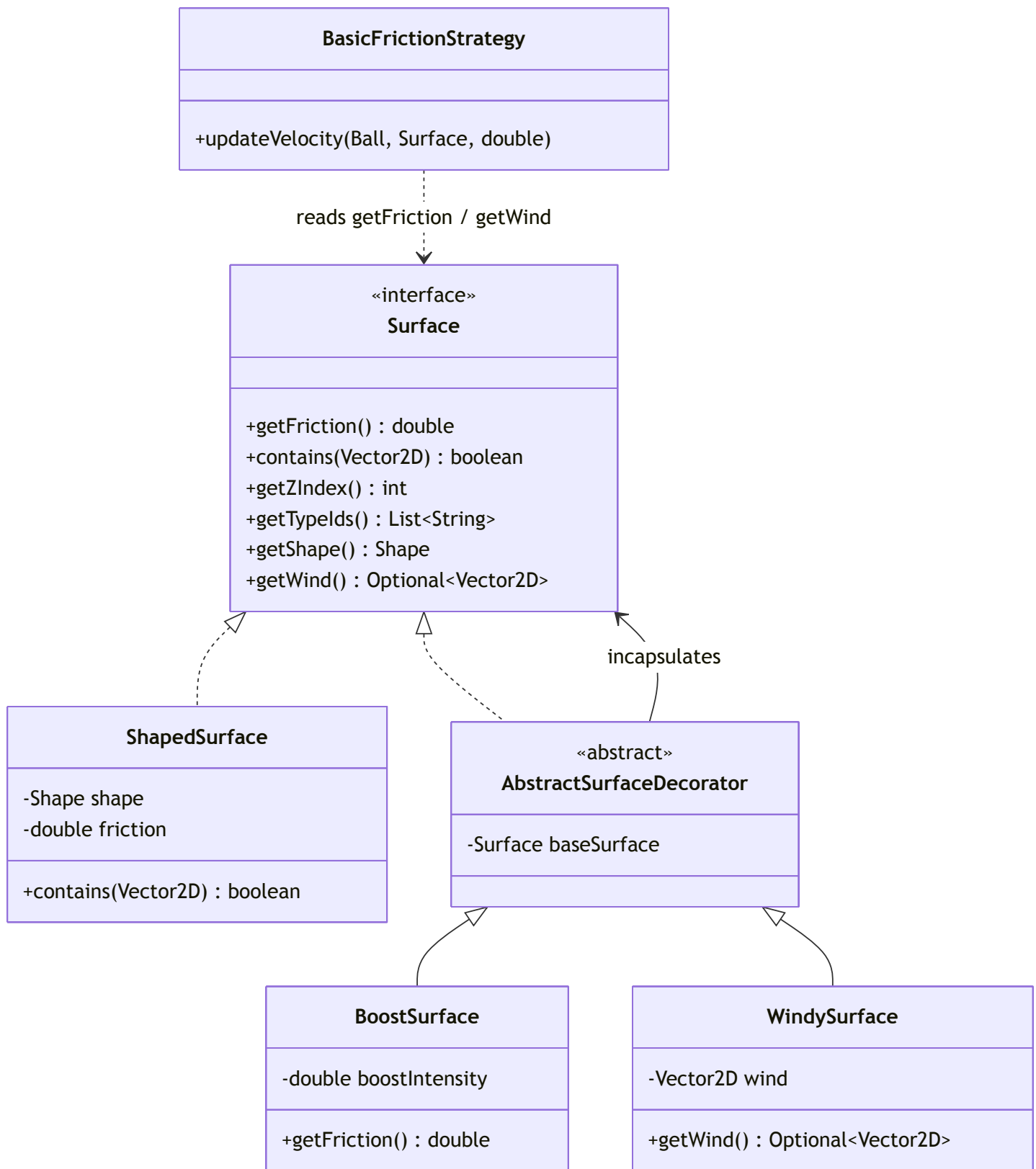


Figura 5: Schema UML del pattern Decorator applicato per l'implementazione delle superfici avanzate.

Pro e Contro

Pro:

- **Componibilità:** è possibile decorare ricorsivamente una stessa superficie con più effetti (es. `BoostSurface` su `WindySurface` su `ShapedSurface`).
- **DRY:** i decorator riutilizzano la logica geometrica e di contenimento della superficie base, sovrascrivendo solo il parametro fisico di loro competenza.
- **Estensibilità:** introdurre un nuovo effetto fisico (es. una superficie che inverte la direzione o applica gravità variabile) richiede solo di creare un nuovo decoratore che estende `AbstractSurfaceDecorator` , senza toccare né `ShapedSurface` né il motore fisico. L'Open/Closed Principle è rispettato per definizione strutturale.

Contro:

- La creazione di superfici composite è verbosa; è necessaria una `SurfaceFactory` per mediare la costruzione.

Fisica della pallina e integrazione Model–Controller

Problema

`PhysicsEngine` (Model) richiede un oggetto `Ball` per leggere posizione/velocità e aggiornarne lo stato ad ogni tick. Lo stato reale della pallina è però custodito nel Controller tramite `BallController` . Esistono due approcci immediati ma chiaramente architetturalmente inesatti:

- duplicare lo stato creando un oggetto `Ball` separato nel Model (rischio di disallineamento);
- far dipendere `PhysicsEngine` da `BallController` (viola la separazione MVC).

Soluzione

Il pattern **Adapter** è implementato dalla classe `BallControllerAdapter` : implementa `Ball` delegando ogni lettura e scrittura direttamente al `BallController` incapsulato.

`PhysicsControllerImpl` istanzia l'adapter ad ogni tick e lo passa a `PhysicsEngine` , che opera su un oggetto `Ball` senza sapere nulla del controller.

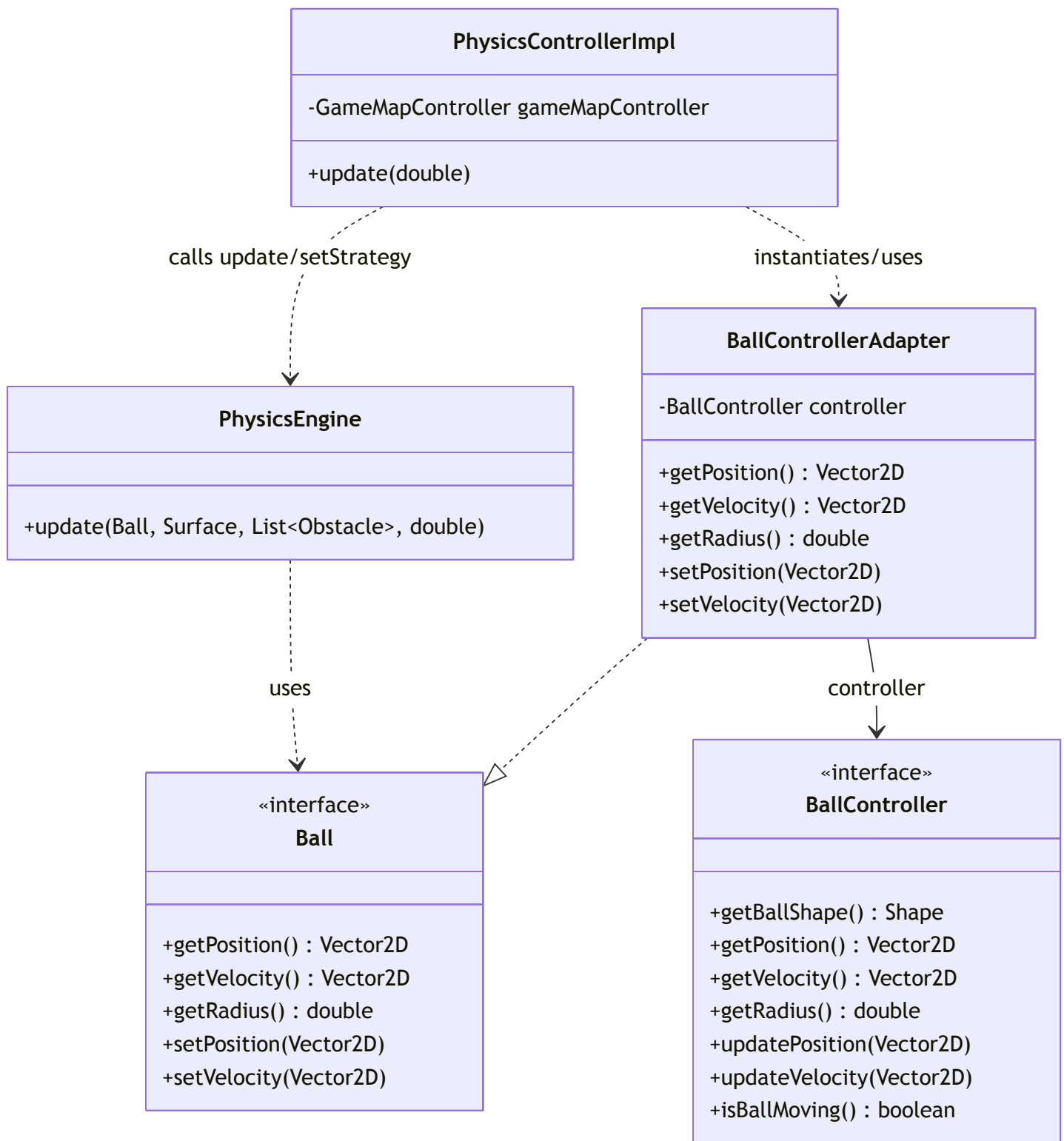


Figura 6: Schema UML del pattern Adapter utilizzato per integrare il motore fisico con la pallina gestita dal controller.

Pro e Contro

Pro:

- **Unica istanza della pallina:** le modifiche del motore fisico si riflettono immediatamente sul controller, senza sincronizzazione esplicita.

- **Indipendenza tra Model e Controller:** `PhysicsEngine` dipende solo da `Ball` ; `BallController` non conosce il Model. Entrambe le interfacce restano indipendenti.
- **Sostituibilità:** se in futuro la pallina venisse gestita da un controller differente, sarà sufficiente creare un nuovo adapter che implementa `Ball` wrapando il nuovo controller, senza alcuna modifica al `PhysicsEngine` .

Contro:

- Aggiunge una classe ponte dedicata, giustificata solo dall'architettura MVC.

2.2.3 Federico Sparvoli

Input del colpo

Problema

Il gioco richiede che il giocatore indichi direzione e potenza del colpo trascinando il mouse dalla pallina. Il sistema deve riconoscere dove inizia il trascinamento, calcolare la direzione e la potenza, mostrare un indicatore visivo, e confermare il colpo solo al rilascio del mouse.

Soluzione

La logica è suddivisa in tre livelli distinti seguendo il pattern MVC:

- Model (`ShotState`): tiene traccia dello stato del colpo in corso. Espone lo stato tramite interfacce strette invece che come oggetto diretto, evitando dipendenze scomode.
- View (`ShotViewPanel` , `ShotListener`): `ShotListener` riceve gli eventi del mouse e li traduce in aggiornamenti sul model. Per disegnare l'indicatore e convertire le coordinate non dipende dalla classe concreta `ShotViewPanel` , ma da due interfacce: `ShotVisualizer` (che definisce come mostrare e confermare il colpo) e `ShotCoordinateConverter` (che definisce come tradurre le coordinate del mouse in coordinate di gioco). Queste due interfacce rappresentano le strategie del pattern Strategy, e `ShotViewPanel` ne è l'implementazione concreta: in questo modo il modo di disegnare o di convertire le coordinate può essere cambiato fornendo una nuova implementazione, senza toccare `ShotListener` .
- Controller (`ShotControllerImpl`): ogni tick interroga il model e, se un colpo è pronto, lo passa alla logica di turno tramite callback, senza memorizzare riferimenti a oggetti mutabili.

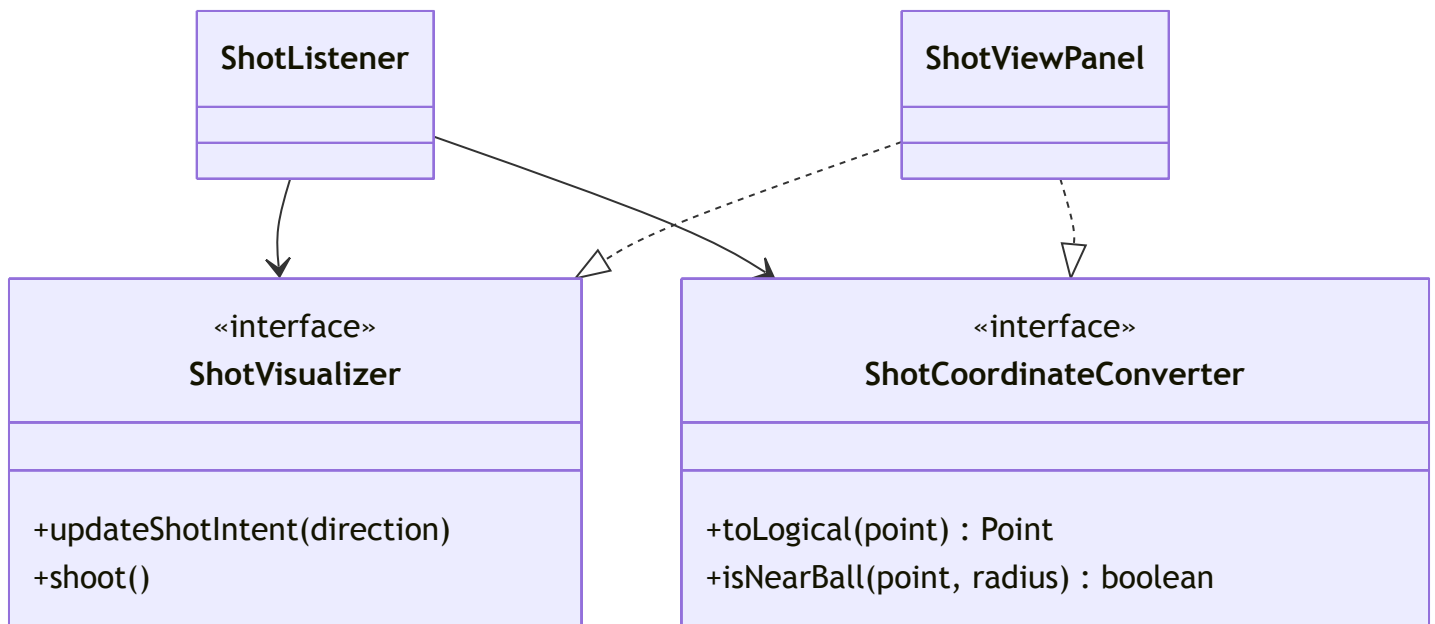


Figura 7: Schema UML del pattern Strategy applicato all'input del colpo.

Pro e Contro

Pro:

Separazione netta tra stato, rendering e coordinamento. Aggiungere un nuovo tipo di indicatore visivo richiede solo una nuova implementazione di `ShotVisualizer`, senza toccare il listener o il controller.

Contro:

La soglia di click sulla pallina è una costante fissa in pixel logici, e non si adatta automaticamente se la pallina cambia dimensione.

Ciclo di vita del match e progressione delle mappe

Problema

Il gioco deve supportare più mappe in sequenza. Quando la pallina entra in buca si avanza alla mappa successiva, se non ce ne sono, si torna al menù. Tutta questa logica non deve stare nel controller principale, che deve rimanere semplice.

Soluzione

La progressione di gioco è gestita da più componenti distinte:

- `MapSequence` (model): tiene la lista delle mappe e sa qual è la corrente. Applica il pattern Factory Method: ogni mappa è prodotta da una `GameMapFactory`, rendendo semplice aggiungerne di nuove.

- **GameMapSequenceFactory** : costruisce la sequenza di mappe tramite un metodo statico (Static Factory Method) che mette sempre la mappa tutorial come prima e mescola le restanti in ordine casuale, così ogni partita è diversa.
- **GameFactory** : costruisce il match per la mappa corrente, collegando tutti i componenti necessari.
- **MatchManager** (controller): usa **GameFactory** per costruire ogni match e gestisce le transizioni tra una mappa e l'altra. Comunica con il resto del sistema solo tramite callback, senza memorizzare oggetti mutabili.

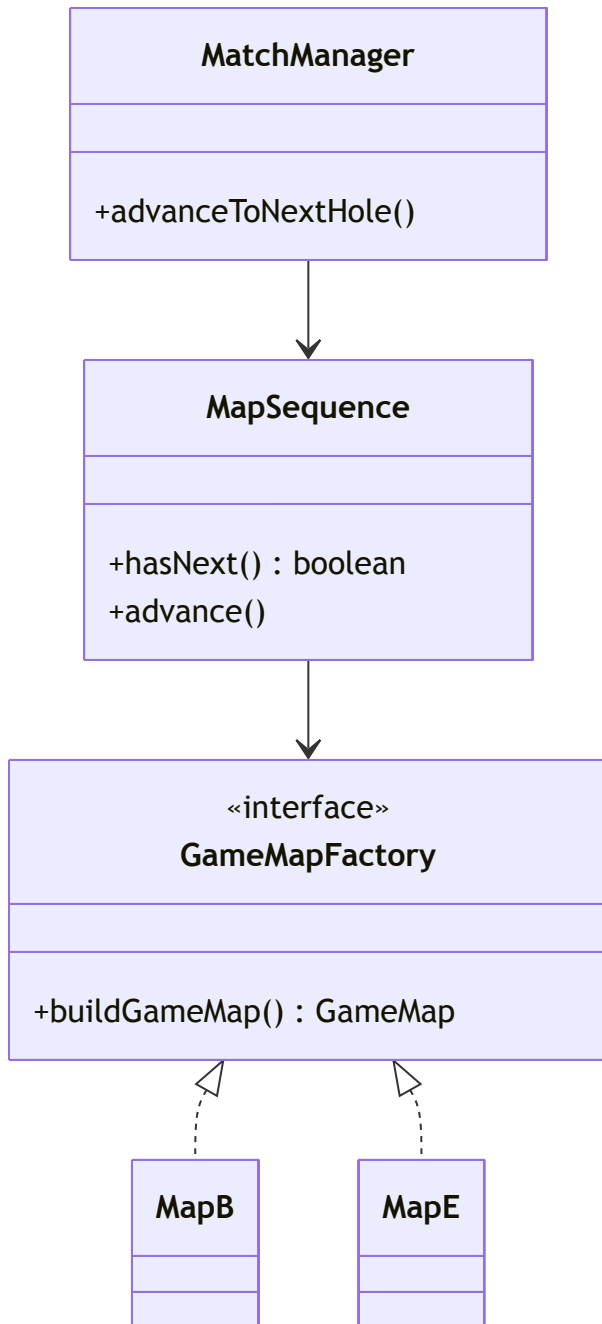


Figura 8: Schema UML del pattern Factory Method per la progressione delle mappe.

Aggiungere una nuova mappa richiede solo di creare una nuova classe e aggiungerla alla sequenza. Il reset al ritorno al menù è automatico.

Contro:

Ad ogni cambio mappa il match viene ricostruito da zero, il che potrebbe rallentare il gioco se le mappe fossero molto complesse.

2.2.4 Mattia D'Ambrosio

Fisica, geometria degli ostacoli e gestione vettoriale

Problema

Gestire le collisioni fisiche tra la pallina e gli ostacoli di forme diverse (rettangoli, cerchi, triangoli) in modo realistico, anche in situazioni critiche come angoli interni tra ostacoli adiacenti o sotto l'effetto di forze esterne, evitando compenetrazioni, vibrazioni della pallina a riposo e rimbalzi innaturali o con direzioni arbitrarie.

Implementare il calcolo vettoriale senza importare librerie esterne pesanti.

Soluzione

La classe `AbstractObstacle` centralizza la logica di calcolo del rimbalzo e del contatto a riposo, evitando ripetizioni di codice nelle sottoclassi. Per i calcoli vettoriali viene utilizzata una classe personalizzata `Vector2D`, ispirata all'omonima classe della libreria Apache Commons Math 3, che fornisce solo i metodi essenziali ai calcoli di gioco, mantenendo l'architettura leggera e priva di dipendenze esterne superflue.

Per rendere i rimbalzi deterministici e privi di direzioni arbitrarie, sono state applicate due strategie:

- Le normali di collisione degli ostacoli sono precalcolate per ogni lato (`WallObstacle`, `TriangleObstacle`) o derivabili geometricamente (`RoundObstacle`).
- Viene sfruttata la profondità di penetrazione per riposizionare la pallina, o parte di essa, fuori dall'ostacolo, prima di calcolare il rimbalzo; in caso di collisioni multiple, si applica la strategia *deepest penetration first*, individuando l'ostacolo con la compenetrazione maggiore, estraendone la pallina e ripetendo il controllo iterativamente finché la pallina non risulta completamente fuori da ogni ostacolo.

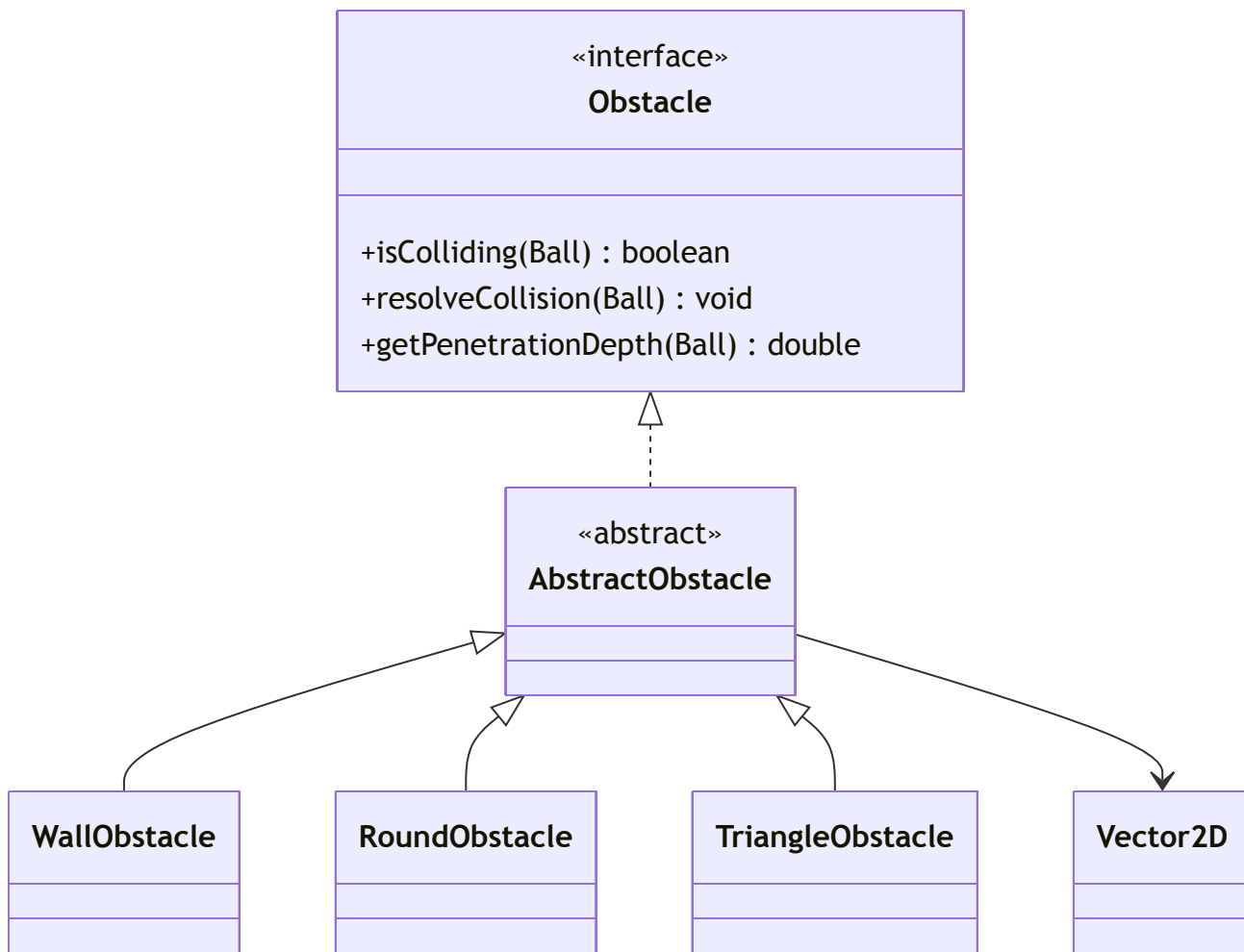


Figura 9: Schema UML della gerarchia degli ostacoli fisici.

Pro e Contro

Pro:

- Fisica altamente stabile e realistica
- Le normali precalcolate riducono drasticamente i calcoli ripetitivi nel game loop (specialmente nei rettangoli).
- La classe personalizzata 'Vector2D' evita di importare tutti i metodi e campi non necessari all'applicazione, alleggerendola
- Architettura scalabile: l'aggiunta di un nuovo ostacolo richiede solo di estendere 'AbstractObstacle' ed implementare il calcolo di compenetrazione.

Contro:

- Se la velocità della pallina in un singolo frame supera lo spessore dell'ostacolo, rischia di attraversarlo (tunneling).
- Il calcolo esatto delle distanze per i triangoli richiede l'estrazione di radici quadrate, operazione che appesantiscono il game loop.

Creazione degli ostacoli avanzati (Appiccicosi, Rimbalzanti e Portali)

Problema

Per gestire le alterazioni di velocità senza moltiplicare le classi, si è introdotto il parametro `bounciness` in `AbstractObstacle`, il quale scala la velocità della pallina al momento del contatto senza alterarne l'angolo di riflessione. Per evitare di creare una nuova classe per ogni ostacolo avanzato, si è sfruttato il constructor chaining: per gli ostacoli base, il costruttore base richiama un secondo costruttore più completo, passandogli un valore di elasticità di default.

Soluzione

Per la gestione dei portali, invece, era fondamentale prevenire la configurazione di stati invalidi nella mappa, come l'esistenza di portali spaati. Per risolvere il problema è stato applicato il pattern `Static Factory Method`. Il costruttore della classe `PortalObstacle` è stato reso privato, delegando l'istanziatura e il collegamento reciproco in memoria al metodo statico `createPair()`, che assicura la creazione di portali sempre e solo a coppie. Infine, per evitare che la pallina rimbalzi all'infinito tra due portali nello stesso frame, è stato implementato un timer di cooldown, che parte al momento dell'uscita della pallina e disattiva momentaneamente il portale ignorando le compenetrazioni.

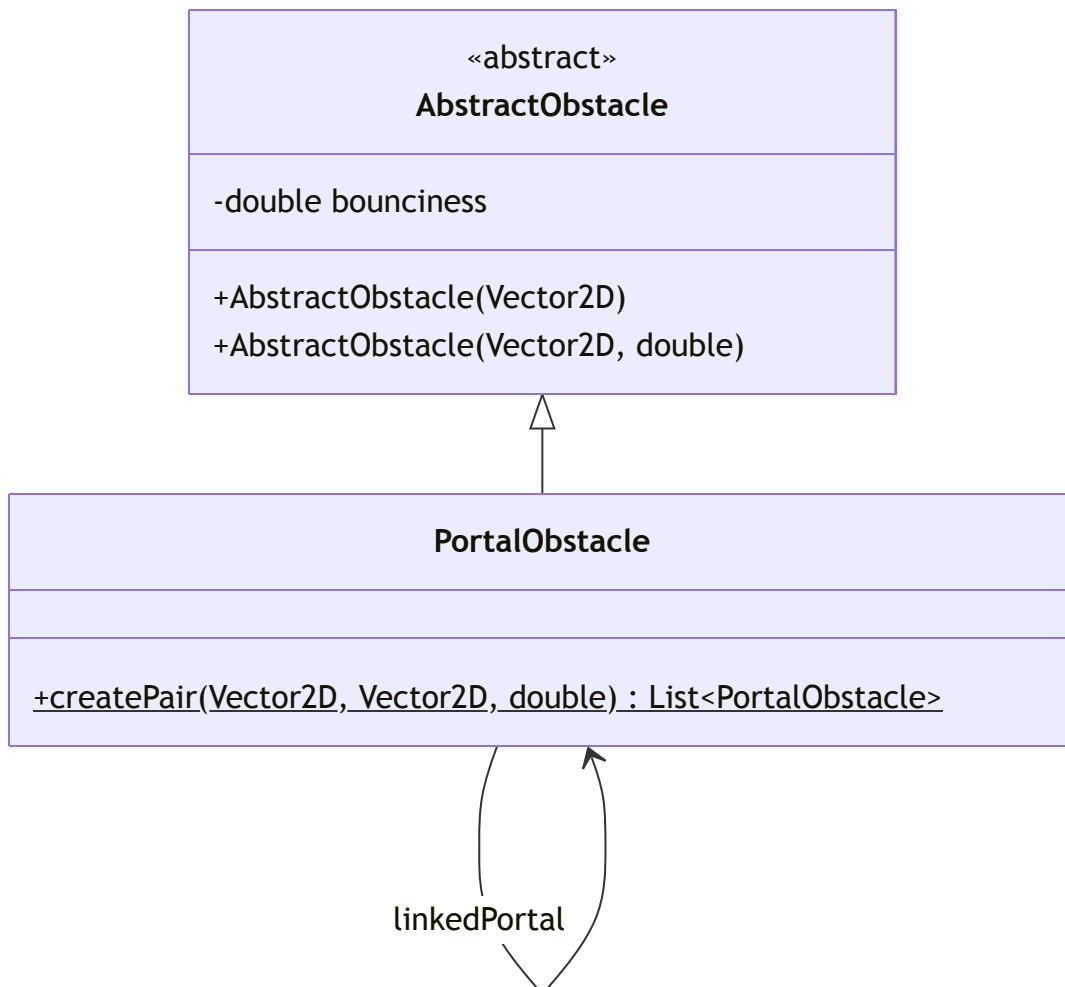


Figura 10: Schema UML del pattern Static Factory Method per i portali e Chaining Constructor per gli ostacoli avanzati.

Pro e Contro

Pro:

- Architettura estremamente flessibile: gli effetti di accelerazione e decelerazione possono essere applicati a qualsiasi forma geometrica tramite un semplice parametro.
- Sicurezza strutturale nella creazione dei portali, che impedisce stati invalidi come la creazione di portali spaiati.

Contro:

- Il cooldown temporale fisso potrebbe scadere prima che la pallina sia uscita dal portale se questa viaggia a una velocità estremamente ridotta, spostandola nell'altro portale nel tiro successivo.

Architettura e separazione Vista-Logica

Problema

La logica di gestione degli ostacoli (collisioni, calcoli fisici, accesso ai dati) deve essere separata dalla rappresentazione grafica e dal resto del gioco, per mantenere un'architettura MVC pulita ed evitare che la vista dipenda direttamente dalle classi o dalle interfacce del modello.

Soluzione

Per mantenere un'architettura MVC pulita e separare nettamente la fisica dalla grafica, il sistema è stato diviso in livelli. L' `ObstacleController` si occupa esclusivamente del modello: gestisce i dati fisici degli `Obstacle` e li passa al motore fisico per risolvere le collisioni.

Per dividere del tutto la vista dal modello, è stato applicato il pattern `Data Transfer Object (DTO)` : il `GameController` fa da mediatore interrogando l' `ObstacleController` e traduce i modelli interni in `ObstacleData` , un record immutabile che contiene solo le informazioni utili per il rendering.

Infine, il `GameController` invia questi dati all'interfaccia `MapElementsView` , implementata da `MapPanel`

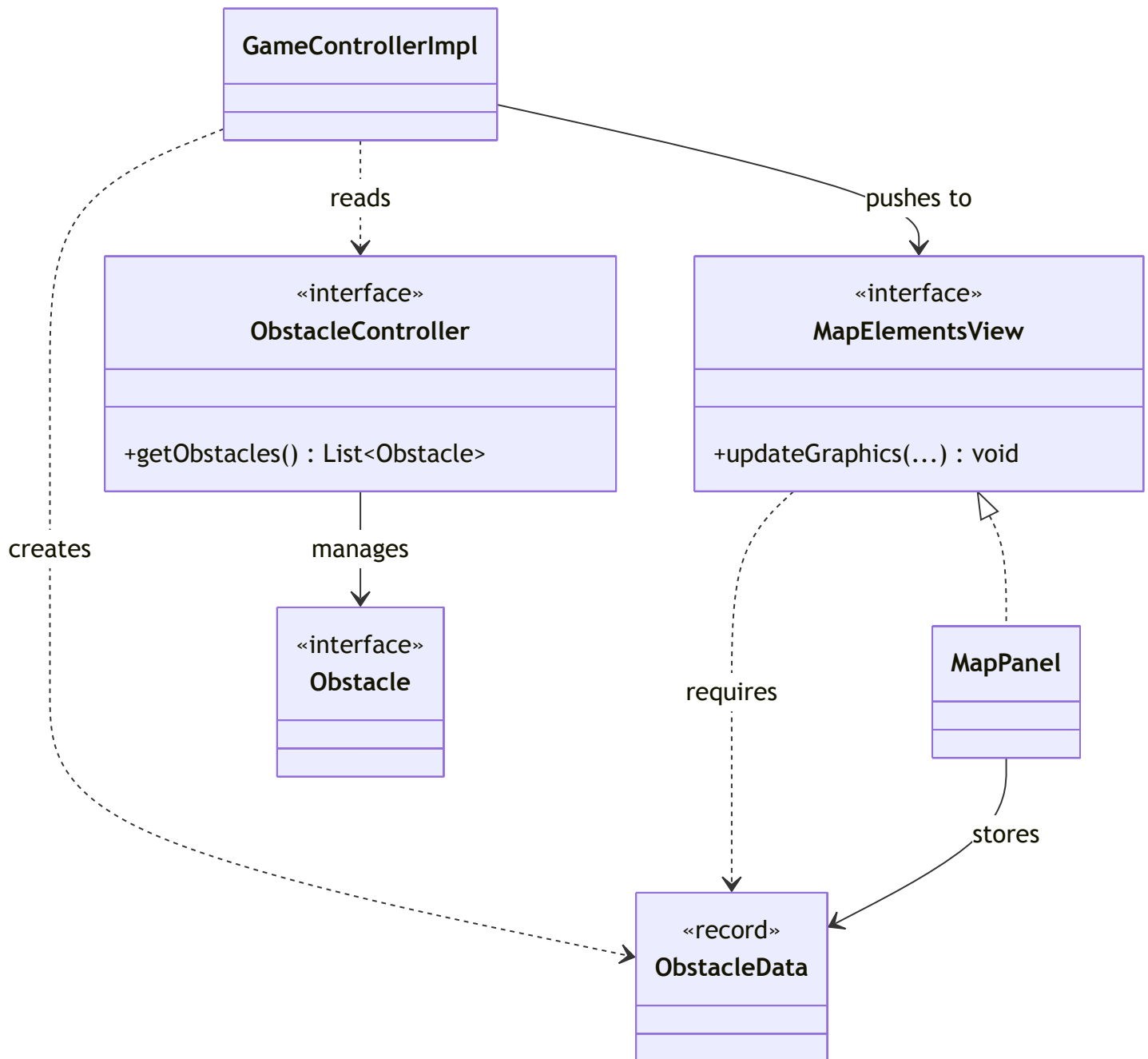


Figura 11: Schema UML del pattern DTO applicato all'architettura MVC.

Pro e Contro

Pro:

- Separazione totale tra modello e vista: la grafica è completamente passiva, riceve i dati già pronti tramite il DTO e si limita a stamparli a schermo. Questo rende il codice molto più facile da mantenere e da testare.
- Tutta la logica decisionale (ad esempio, capire come colorare un ostacolo o se si tratta di un portale) rimane chiusa in modo sicuro nel controller, sollevando la vista dal dover fare ragionamenti sui tipi di oggetti.

Contro:

- Avere tutti questi strati intermedi aumenta parecchio il numero di interfacce, record e file nel progetto, rendendo l'architettura iniziale un po' pesante da configurare.
- Creare una lista nuova di record `ObstacleData` a ogni singolo frame per il rendering significa allocare e scartare continuamente oggetti in memoria, causando un overhead prestazionale inevitabile per il Garbage Collector.

Capitolo 3: Sviluppo

3.1 Testing automatizzato

3.1.1 Daniel Patryk Bak

I componenti testati riguardano la fabbricazione dell'interfaccia utente, l'integrità delle risorse multimediali e la persistenza dei dati:

- **UIFactoryTest:** Verifica che tutti i componenti Swing (`JButton`, `JLabel`, `TextField`) generati dalla factory rispettino le specifiche di design imposte dal gioco, garantendo l'assegnazione corretta dei font personalizzati, delle dimensioni predefinite, dei colori e dei bordi.
- **ResourcePresenceTest:** Assicura l'integrità dell'applicativo, verificando che tutte le risorse statiche vitali per il funzionamento dell'interfaccia (font, soundtrack, texture delle superfici,...) siano presenti e caricabili correttamente.
- **LeaderBoardManagerTest:** Controlla la logica di aggiornamento e scrittura su file della classifica globale. Tramite un ambiente pulito (creazione e cancellazione del file ad ogni test), verifica la regola di dominio fondamentale: il punteggio storico di un giocatore viene sovrascritto solo se il nuovo punteggio ottenuto è strettamente inferiore (migliore) del precedente.

3.1.2 Giacomo Mengozzi

I componenti testati riguardano la pallina, la mappa e le superfici speciali:

- **BallImplTest:** Inizializzazione dello stato (posizione, raggio, velocità), correttezza dei setter e integrazione con `PhysicsEngine / BasicFrictionStrategy` per verificare che l'attrito azzeri la velocità; testa inoltre il lancio di `IllegalStateException` per posizioni fuori mappa.

- **GameMapImplTest:** Rifiuto di argomenti `null` nel costruttore e comportamento di `getSurfaceAt` (restituzione della superficie corretta, priorità per z-index più alto, eccezione per punto fuori dai limiti).
- **BoostSurfaceTest:** Validazione del costruttore, calcolo dell'attrito negativo, delega delle proprietà alla superficie base e integrazione con `PhysicsEngine` (accelerazione della pallina in moto, speed cap).
- **WindySurfaceTest:** Validazione del costruttore, correttezza del vettore di vento per direzione, delega delle proprietà e composizione con `BoostSurface` (decorator su decorator).

3.1.3 Federico Sparvoli

I componenti testati sono `ShotState`, `GameState` e `GameFactory`.

- **ShotStateTest:** controlla che lo stato del tiro funzioni bene: all'inizio è vuoto, c'è una potenza minima, la potenza massima non si può superare, si può consumare un colpo e si può resettare tutto.
- **GameStateTest:** controlla la logica dei turni: come si crea, come si passa al turno successivo (e si ricomincia da capo dopo l'ultimo), quanti colpi sono stati fatti, e che i colpi troppo deboli o fatti mentre la pallina si muove vengano ignorati.
- **GameFactoryTest:** controlla che la partita creata dalla factory parta sempre in uno stato iniziale corretto su entrambe le mappe: tutti i controller devono essere presenti, il primo giocatore è quello giusto, e lo stato del tiro è vuoto.

3.1.4 Mattia D'Ambrosio

I componenti testati riguardano la parte logico-matematica e la gestione degli ostacoli all'interno del sistema.

- **Vector2DTest:** Verifica la correttezza di tutte le operazioni algebriche fondamentali sui vettori custom (somma, sottrazione, prodotto scalare, normalizzazione e distanza), controllando la corretta gestione dei limiti di precisione nei calcoli floating-point tramite la tolleranza `EPSILON`.
- **ObstacleCollisionTest:** Verifica l'accuratezza del calcolo delle collisioni e della profondità di penetrazione per `WallObstacle`, `RoundObstacle` e `TriangleObstacle`. Testa i casi limite, come l'impatto sugli spigoli vivi e l'espulsione fisica della pallina quando si trova sia all'esterno che completamente all'interno dei solidi.
- **ObstacleControllerTest:** Assicura il corretto funzionamento del controller come intermediario. Verifica che la lista dei modelli matematici (`Obstacle`) e delle forme grafiche (`Shape`) siano popolate coerentemente e che il motore fisico riceva i dati corretti per la risoluzione dei contatti in ogni frame.

3.2 Note di sviluppo

3.2.1 Daniel Patryk Bak

Factory

Usata nella classe `UserInterfaceFactory` per poter creare elementi grafici come bottoni, etichette tutte uniformate.

Permalink: <https://github.com/jjacomo/OOP25-mini-gOOf/blob/a03bd528fdd870c74b8574f68c74972313d66720/src/main/java/it/unibo/minigolf/view/elements/UserInterfaceFactory.java#L1>

Lamda Expressions

Utilizzate in vari punti un esempio al permalink.

Permalink: <https://github.com/jjacomo/OOP25-mini-gOOf/blob/a03bd528fdd870c74b8574f68c74972313d66720/src/main/java/it/unibo/minigolf/view/panels/NewGamePanel.java#L96>

...

3.2.2 Giacomo Mengozzi

Java Records

Permalink: <https://github.com/jjacomo/OOP25-mini-gOOf/blob/a03bd528fdd870c74b8574f68c74972313d66720/src/main/java/it/unibo/minigolf/util/shapes/Circle.java#L17>

Java Reflection API

Permalink: <https://github.com/jjacomo/OOP25-mini-gOOf/blob/5f753c229d34960fe24214219c043168836aa14d/src/main/java/it/unibo/minigolf/view/panels/MapPanel.java#L154>

Stream API e lambda expressions

Permalink: <https://github.com/jjacomo/OOP25-mini-gOOf/blob/a03bd528fdd870c74b8574f68c74972313d66720/src/main/java/it/unibo/minigolf/model/map/GameMapImpl.java#L54>

Optional

Permalink: <https://github.com/jjacomio/OOP25-mini-gOOf/blob/a03bd528fdd870c74b8574f68c74972313d66720/src/main/java/it/unibo/minigolf/model/surfaces/Surface.java#L60>

<https://github.com/jjacomio/OOP25-mini-gOOf/blob/a03bd528fdd870c74b8574f68c74972313d66720/src/main/java/it/unibo/minigolf/model/physics/velocity/BasicFrictionStrategy.java#L67>

Utilizzo della libreria SLF4J

Ad esempio in <https://github.com/jjacomio/OOP25-mini-gOOf/blob/a03bd528fdd870c74b8574f68c74972313d66720/src/main/java/it/unibo/minigolf/model/physics/PhysicsEngine.java#L62>

3.2.3 Federico Sparvoli

Lambda expressions e method reference

Permalink: <https://github.com/jjacomio/OOP25-mini-gOOf/blob/f616e830c3ba78c84ab7d850d5690ee1ea46ccb0/src/main/java/it/unibo/minigolf/controller/game/GameControllerImpl.java#L132>

Java Records

Permalink: <https://github.com/jjacomio/OOP25-mini-gOOf/blob/e5ae0bf0fd17ad5844e7f8de9e0d114436baf99f/src/main/java/it/unibo/minigolf/model/save/SaveData.java#L21>

Stream API

Permalink: <https://github.com/jjacomio/OOP25-mini-gOOf/blob/e5ae0bf0fd17ad5844e7f8de9e0d114436baf99f/src/main/java/it/unibo/minigolf/model/logic/GameState.java#L43>

Optional

Permalink: <https://github.com/jjacomio/OOP25-mini-gOOf/blob/e5ae0bf0fd17ad5844e7f8de9e0d114436baf99f/src/main/java/it/unibo/minigolf/model/logic/ShotState.java#L72>

Libreria Gson (com.google.code.gson)

Permalink: <https://github.com/jjacomio/OOP25-mini->

[gOOLF/blob/e5ae0bf0fd17ad5844e7f8de9e0d114436baf99f/src/main/java/it/unibo/minigolf/model/save/SaveManager.java#L21](https://github.com/jjacomo/OOP25-mini-gOOLF/blob/e5ae0bf0fd17ad5844e7f8de9e0d114436baf99f/src/main/java/it/unibo/minigolf/model/save/SaveManager.java#L21)

3.2.4 Mattia D'Ambrosio

Functional Interface

Permalink: <https://github.com/jjacomo/OOP25-mini-gOOLF/blob/4ca2f5efea4af39cd1070ba6f7cdf040f713e59f/src/main/java/it/unibo/minigolf/controller/obstaclecontroller/ObstacleController.java#L11>

Collezioni Immutabili

Permalink: <https://github.com/jjacomo/OOP25-mini-gOOLF/blob/4ca2f5efea4af39cd1070ba6f7cdf040f713e59f/src/main/java/it/unibo/minigolf/controller/obstaclecontroller/ObstacleControllerImpl.java#L23>

Constructor Chaining

Permalink: <https://github.com/jjacomo/OOP25-mini-gOOLF/blob/4ca2f5efea4af39cd1070ba6f7cdf040f713e59f/src/main/java/it/unibo/minigolf/model/obstacles/AbstractObstacle.java#L50>

Lambda Expressions

Permalink: <https://github.com/jjacomo/OOP25-mini-gOOLF/blob/4ca2f5efea4af39cd1070ba6f7cdf040f713e59f/src/test/java/it/unibo/minigolf/util/Vector2DTest.java#L29>

Capitolo 4: Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Daniel Patryk Bak

Questa esperienza è stata molto interessante poiché tocca non solo aspetti di pura scrittura di codice, ma bensì richiede le capacità di potersi relazionare in un gruppo, aiutarsi a vicenda e soprattutto organizzare cosa fare e in che tempi. Per questo sono molto contento di aver lavorato con piacere con i miei colleghi, perchè tutto questo è stato rispettato. Per poter raggiungere questi "principi", abbiamo deciso di scrivere (oltre a importanti informazioni contenute nei commit) delle

note ogni volta che si metteva mano al progetto, oltre a fare delle piccole riunioni periodiche. Abbiamo inoltre dedicato molto tempo alla fase di progettazione, prima ancora di fare la proposta ufficiale, e questo ha aiutato poi a realizzare senza troppi problemi il progetto concreto.

Per quanto riguarda l'aspetto più pratico/tecnico del progetto, sono contento di aver individuato correttamente, e poi implementato, il metodo di transizione tra i vari pannelli, e aver adibito ad ognuno un compito specifico. La leaderboard, per quanto semplice, è molto soddisfacente da vedere. La partita multiplayer invece non è stata particolarmente difficile da realizzare, grazie ad una già buona base fornita dal mio collega Federico, per quanto riguarda la logica di una partita.

Ne ho già parlato con i miei colleghi e spero che in futuro aggiorneremo il gioco (con tempi molto più "rilassati") qualche feature che non è per niente funzionale, ma che aggiunge quel tocco in più, come altre feature audio, aspetti grafici avanzati, ostacoli e superfici nuove, mappe disegnate in modo migliore. Le idee sono tante, e sono contento che le basi siano abbastanza solide da poterci ancora lavorare su.

4.1.2 Giacomo Mengozzi

Sono molto soddisfatto del contributo fornito al progetto, non soltanto per il lavoro realizzato, ma soprattutto per l'esperienza maturata durante l'intero percorso di sviluppo. Il tempo e le energie investite sono stati considerevolmente superiori alle aspettative iniziali, ma ritengo che siano stati impiegati in modo proficuo.

Dal punto di vista tecnico, mi sono occupato dell'implementazione delle superfici, della mappa e della buca, nonché della loro integrazione nelle mappe di gioco, coordinando l'interazione tra questi elementi secondo le leggi della fisica.

Le difficoltà maggiori hanno riguardato la comprensione e l'applicazione del pattern architetturale MVC nei suoi tre livelli, in particolare per quanto concerne la comunicazione tra le rispettive classi. A queste si è aggiunta la sfida di approcciare il progetto partendo da un'esperienza di programmazione quasi nulla, il che ha reso la fase iniziale particolarmente impegnativa.

Con il progredire del lavoro e grazie all'aiuto e supporto dei miei compagni, tuttavia, la padronanza degli strumenti e delle pratiche di sviluppo è cresciuta progressivamente, consentendo di scrivere codice con maggiore consapevolezza.

In particolare, sono molto contento della costante comunicazione tra i miei compagni che c'è stata durante tutto il percorso di sviluppo, e del modo in cui siamo riusciti a unire insieme le nostre parti di codice per arrivare alla versione attuale del progetto.

In futuro, mi piacerebbe continuare ad arricchire il progetto con nuove feature secondarie come l'aggiunta di effetti audio della pallina per rendere ancora più personale questo gioco a cui sono ormai più che affezionato.

4.1.3 Federico Sparvoli

Il mio contributo principale riguarda il sistema di input del colpo, la gestione del ciclo di vita dei match e il sistema di salvataggio. Sono soddisfatto della separazione raggiunta tra model, view e controller, in particolare dell'eliminazione di tutti i warning SpotBugs senza ricorrere a `@SuppressWarnings`, ottenuta tramite interfacce strette e callback funzionali.

Una scelta progettuale che ho fatto è stata quella di rendere synchronized i metodi pubblici di `ShotState` e di dichiarare volatile il campo che segnala se la pallina è in movimento in `GameState`. Attualmente, gli aggiornamenti del gioco e la gestione dell'input, avvengono in modo sequenziale, all'interno dello stesso gameloop, quindi queste precauzioni non sono necessarie, ma le ho comunque inserite per rendere le classi più robuste, qualora in futuro il gioco venisse esteso con i thread.

Un aspetto migliorabile è la soglia di click sulla pallina (`CLICK_RADIUS`), che al momento è un numero fisso di pixel e non cambia se la pallina viene ingrandita o rimpicciolita. In futuro sarebbe meglio calcolarlo in base al raggio vero della pallina.

Per quanto riguarda il cambio delle mappe, al momento ricostruiamo da capo tutto il controller del gioco ad ogni nuova mappa. Con mappe molto grandi questo potrebbe rallentare il gioco. Il prossimo passo sarebbe caricare le mappe in anticipo, senza bloccare il gioco.

Il sistema di salvataggio memorizza la partita in formato JSON usando Gson. Salva solo il numero della mappa, non tutta la geometria. Così i file sono piccoli e il salvataggio non dipende da come sono fatte le mappe dentro. Un limite attuale è che il salvataggio viene cancellato appena lo si carica, quindi non si può riprendere la stessa partita due volte senza salvarla di nuovo.

4.1.4 Mattia D'Ambrosio

Il mio contributo principale ha riguardato l'architettura geometrico-matematica degli ostacoli, lo sviluppo della classe vettoriale dedicata e l'implementazione della fisica delle collisioni e degli elementi avanzati (ostacoli elastici e portali). Sono pienamente soddisfatto della stabilità raggiunta nel calcolo delle penetrazioni e dell'algoritmo di Normal Blending per la risoluzione deterministica degli angoli interni, che ha rimosso qualsiasi arbitrarietà fisica. Dal punto di vista architetturale, l'introduzione dell' `ObstacleController` unita all'uso del pattern DTO (`ObstacleData`) ha garantito

un disaccoppiamento MVC totale, permettendo la realizzazione di una vera e propria Vista Passiva per il rendering grafico.

Un aspetto decisamente migliorabile riguarda il fenomeno del tunneling: se la pallina si muove a una velocità talmente elevata da superare lo spessore di un ostacolo sottile in un singolo frame, il motore manca la collisione attraversando l'oggetto. In futuro, per ovviare a questo problema, si potrebbe implementare un algoritmo di Continuous Collision Detection (CCD) basato sul raycasting o sul campionamento della traiettoria.

Un altro limite risiede nel meccanismo di cooldown temporale dei portali; sebbene la soglia fissa a 1000ms sia efficace nella maggior parte dei contesti, se la pallina entra in un portale a velocità quasi nulla rischia di rimanere ferma sulla destinazione oltre la scadenza del timer, riattivando il portale.

Un'evoluzione futura prevedrebbe un'immunità basata sulla geometria, disattivando il cooldown solo quando la pallina ha fisicamente interrotto la collisione con il portale di arrivo.

4.2 Difficoltà incontrate e commenti per i docenti

Daniel Patryk Bak: Per quanto abbia veramente apprezzato questo progetto, (vedere nascere da zero un videogioco è uno dei miei piccoli sogni che porto sin da bambino) credo che come esame porti via veramente tanto (troppo) tempo. Capisco anche che per un progetto del genere è necessario, (se avessimo voluto aggiungere qualche aspetto accessorio anche banale sicuramente avremmo raggiunto le 100 ore), infatti ad un certo punto abbiamo dovuto frenare l'entusiasmo e cercare di fare soltanto ciò che era veramente necessario (*in realtà lo voluto anche come un aspetto interessante per progetti veri e propri che hanno scadenze da rispettare e quindi inevitabili "cut content"*).

Questo corso, per la complessità che presenta dal punto di vista teorico e poi pratico, credo debba essere scorporato in due: uno da 6 CFU, dove fare una introduzione al linguaggio e la sua evoluzione, quindi un altro da 12 CFU dove poter parlare esclusivamente dell'aspetto di progettazione e dell'utilizzo dei pattern, con quindi il progetto, che ridimensionerei a circa 50 ore a testa. È inevitabile che questo progetto porti via del tempo agli altri esami, poi, è vero che è in mano allo studente la capacità di sapersi organizzare, però ho percepito parecchio il fatto di aver quel "qualcosa in più" da fare durante la settimana. Come gruppo sono abbastanza certo che abbiamo praticamente lavorato ogni settimana, almeno 5 ore, quindi è stato come avere un corso in più a cui porre attenzione. Per quanto poi non nascondo che a volte è stato veramente piacevole, ma come ogni progetto ha anche aspetti inevitabilmente noiosi e certi imprevisti posso risultare spiacevoli.

Tutto sommato però, per quanto appunto faticoso dal punto di vista del tempo impiegato, credo sia un'esperienza molto positiva, e che preannuncia un possibile vero lavoro futuro.

Appendice A

Guida Utente

- **Menù Iniziale:**

Avviando il gioco viene mostrato il menù principale dal quale è possibile iniziare una nuova partita premendo il tasto `PLAY` ; bisognerà poi scegliere il numero di giocatori, premere il tasto `OK` ed inserirne i nomi. Infine, premere `START MATCH` . Se è presente una partita salvata, il gioco chiederà se la si vuole caricare o iniziarne una nuova. Dal menù è inoltre possibile consultare la classifica finale delle partite precedenti tramite il tasto `LEADERBOARD` .

- **Controlli In-Game:**

Per effettuare un colpo è necessario cliccare e tenere premuto il mouse sulla pallina, per poi trascinare nella direzione opposta a quella in cui si vuole colpire. Più lungo è il trascinamento, più potente sarà il colpo. Rilasciare il mouse per confermare il colpo.

Un indicatore colorato mostra la direzione e la potenza: verde per colpi deboli, giallo per colpi medi, rosso per colpi forti.

- **Regole:**

Ogni giocatore ha a disposizione al massimo 7 colpi per buca. Se non si entra in buca entro il limite, il turno passa al giocatore successivo (o, in caso di singleplayer, si passa alla mappa successiva). Il punteggio di ogni giocatore corrisponde al numero di colpi effettuati su ciascuna mappa e meno colpi si fanno, meglio è. Al termine di ogni mappa viene mostrata una classifica intermedia e una volta completate tutte le mappe, il punteggio finale viene salvato nella classifica globale.

- **Pausa:**

Durante il gioco è possibile mettere in pausa premendo il tasto `ESC`, purché la pallina non sia in movimento. Dal menù di pausa è possibile riprendere la partita, saltare la mappa attuale, consultare il tutorial o tornare al menù principale.

Appendice B

Esercitazioni di laboratorio

B.0.1 federico.sparvoli@studio.unibo.it

- Laboratorio 06: <https://github.com/fedesparvo1-a11y/lab06>